

# Demonstration of Space Complexity Using C

Name: ARITRA TALUKDAR

Roll No: CSB24034

## Purpose

This report demonstrates constant, linear, and quadratic auxiliary space complexities using simple C programs. Only additional memory allocated by the algorithms is considered.

## Algorithms Demonstrated

- **O(1) Space:** Finding the maximum element in an array using scalar variables.
- **O(n) Space:** Reversing an array using an auxiliary array.
- **O(n<sup>2</sup>) Space:** Creating an adjacency matrix for a graph.

## Source Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 /* -----
5  CONSTANT SPACE ALGORITHM -> O(1)
6  Find maximum element in an array, no extra space is used.
7  ----- */
8 int constantSpaceMax(int arr[], int n, size_t *auxMem)
9 {
10     int max = arr[0];
11
12     *auxMem = 0; // no auxiliary memory
13
14     for (int i = 1; i < n; i++)
15         if (arr[i] > max)
16             max = arr[i];
17
18     return max;
```

```

19 }
20
21 /* -----
22  LINEAR SPACE ALGORITHM -> O(n)
23  Reverse array using extra array, that extra array is the
24  auxiliary space used.
25 ----- */
26 void linearSpaceReverse(int arr[], int n, size_t *auxMem)
27 {
28     int *rev = (int *)malloc(n * sizeof(int));
29     if (!rev)
30         return;
31
32     *auxMem = n * sizeof(int); // auxiliary array
33
34     for (int i = 0; i < n; i++)
35         rev[i] = arr[n - i - 1];
36
37     free(rev);
38 }
39
40 /* -----
41  QUADRATIC SPACE ALGORITHM -> O(n^2)
42  Graph adjacency matrix. The matrix itself is the auxiliary
43  space used,
44  which grows quadratically with n.
45 ----- */
46 void quadraticSpaceGraph(int n, size_t *auxMem)
47 {
48     int **adj = (int **)malloc(n * sizeof(int *));
49     if (!adj)
50         return;
51
52     for (int i = 0; i < n; i++)
53         adj[i] = (int *)malloc(n * sizeof(int));
54
55     *auxMem = (n * sizeof(int *)) + (n * n * sizeof(int));
56
57     /* Initialize matrix */
58     for (int i = 0; i < n; i++)
59         for (int j = 0; j < n; j++)

```

```

58         adj[i][j] = (i == j) ? 0 : 1;
59
60     for (int i = 0; i < n; i++)
61         free(adj[i]);
62     free(adj);
63 }
64
65 int main()
66 {
67     int n;
68     size_t auxMem;
69
70     printf("Enter number of elements: ");
71     scanf("%d", &n);
72
73     int *arr = (int *)malloc(n * sizeof(int));
74     for (int i = 0; i < n; i++)
75         arr[i] = i + 1;
76
77     /* ----- CONSTANT SPACE ----- */
78     printf("\n==== CONSTANT SPACE =====\n");
79     printf("Maximum Element: %d\n",
80           constantSpaceMax(arr, n, &auxMem));
81     printf("Auxiliary Memory Used: %zu bytes\n", auxMem);
82     printf("Space Complexity: O(1)\n");
83
84     /* ----- LINEAR SPACE ----- */
85     printf("\n==== LINEAR SPACE =====\n");
86     linearSpaceReverse(arr, n, &auxMem);
87     printf("Auxiliary Memory Used: %zu bytes\n", auxMem);
88     printf("Space Complexity: O(n)\n");
89
90     /* ----- QUADRATIC SPACE ----- */
91     printf("\n==== QUADRATIC SPACE =====\n");
92     quadraticSpaceGraph(n, &auxMem);
93     printf("Auxiliary Memory Used: %zu bytes\n", auxMem);
94     printf("Space Complexity: O(n^2)\n");
95
96     free(arr);
97     return 0;
98 }

```

# Program Output

The following screenshots show the combined output of all three algorithms executed in a single program run.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 /* ----- CONSTANT SPACE ALGORITHM ----- */
5 /* CONSTANT SPACE ALGORITHM -> O(1)
6 Find maximum element in an array, no extra space is used.
7 ----- */
8 int constantSpaceMax(int arr[], int n, size_t *auxMem)
9 {
10     int max = arr[0];
11
12     *auxMem = 0; // no auxiliary memory
13
14     for (int i = 1; i < n; i++)
```

```
PS C:\Users\HP\LEPTOP\OneDrive\Desktop\Advanced prog> ./disp
===== CONSTANT SPACE =====
Maximum Element: 100
Auxiliary Memory Used: 0 bytes
Space Complexity: O(1)

===== LINEAR SPACE =====
Auxiliary Memory Used: 400 bytes
Space Complexity: O(n)

===== QUADRATIC SPACE =====
Auxiliary Memory Used: 40800 bytes
Space Complexity: O(n^2)
PS C:\Users\HP\LEPTOP\OneDrive\Desktop\Advanced prog> ./disp
```

Figure 1: 1st sample output

```
65 int main()
66 {
67     constantsSpaceMax(arr, n, &auxMem);
68     printf("Auxiliary Memory Used: %zu bytes\n", auxMem);
69     printf("Space Complexity: O(1)\n");
70
71     /* ----- LINEAR SPACE ----- */
72     printf("\n===== LINEAR SPACE =====\n");
73     linearSpaceReverse(arr, n, &auxMem);
74     printf("Auxiliary Memory Used: %zu bytes\n", auxMem);
75     printf("Space Complexity: O(n)\n");
76
77     /* ----- QUADRATIC SPACE ----- */
78     printf("\n===== QUADRATIC SPACE =====\n");
79     quadraticSpaceGraph(n, &auxMem);
80     printf("Auxiliary Memory Used: %zu bytes\n", auxMem);
81 }
```

```
PS C:\Users\HP\LEPTOP\OneDrive\Desktop\Advanced prog> ./disp
===== CONSTANT SPACE =====
Maximum Element: 500
Auxiliary Memory Used: 0 bytes
Space Complexity: O(1)

===== LINEAR SPACE =====
Auxiliary Memory Used: 2000 bytes
Space Complexity: O(n)

===== QUADRATIC SPACE =====
Auxiliary Memory Used: 1004000 bytes
Space Complexity: O(n^2)
PS C:\Users\HP\LEPTOP\OneDrive\Desktop\Advanced prog>
```

Figure 2: 2nd sample output

## Conclusion

The program demonstrates how auxiliary space grows with different algorithm designs. Constant space algorithms use only scalar variables, linear space algorithms allocate

one additional array, and quadratic space algorithms require two-dimensional memory allocation.